

Build a Production-Ready Code Snippet Manager with Monaco Editor and React

[Back](#)

Mandatory

Domain

Frontend Development

Difficulty

Hard

Industries

Technology

Skills

- Artificial Intelligence
- Configuration Management
- Frontend Development
- Solution Architecture
- API Consumption
- Local Storage API
- UI Design

Tools

- Docker
- Monaco Editor
- Nginx
- React
- Redux
- Tailwind CSS
- Zustand



Pending Submission

Please submit your work when ready.

Deadline: **30 May 2026, 04:59 pm**

TIME REMAINING

52m

Overview

Instructions

Resources

Submit

Objective



Report Issue

You will build a feature-rich code snippet management application using React and the Monaco Editor. You'll learn to integrate a

partnr

[Learn More](#)

[Get Started](#)

that mirrors real-world developer productivity applications.

Description

Background

In modern software development, productivity tools are essential. Code snippet managers help developers save, find, and reuse common pieces of code, eliminating repetitive typing and reducing errors. Integrating a full-featured code editor like Monaco Editor (the editor that powers VS Code) into a web application provides a superior user experience with features like syntax highlighting, auto-indentation, and theme support.

This project challenges you to build such a tool. You will leverage a modern frontend stack, focusing on component-based architecture with React, robust state management, and interaction with browser APIs like Local Storage and the Clipboard API. You will also integrate with the GitHub Gist API, a common requirement in developer-focused applications. The entire application will be containerized using Docker, ensuring a consistent development and evaluation environment, a standard practice in professional software engineering.

Implementation Details

Follow these steps to build the application. You have the freedom to structure your components and state as you see fit, as long as the core requirements are met.

Step 1: Project Setup

Initialize a new React application using your preferred toolchain (e.g., Create React App, Vite). Install the necessary dependencies, including `react`, `react-dom`, `@monaco-editor/react`, a routing library like `react-router-dom`, a state management library, and a CSS framework like Tailwind CSS.

Step 2: Containerization with Docker

Create a `Dockerfile` and `docker-compose.yml` at the root of your project to serve the application. This ensures your application can be

partnr

[Learn More](#)

[Get Started](#)

```
# docker-compose.yml
version: '3.8'
services:
  web:
    build: .
    ports:
      - '3000:80' # Expose port 3000, mapped to port 80 in the container
    environment:
      - NODE_ENV=production
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost"] # Basic health check
      interval: 30s
      timeout: 10s
      retries: 3
```

Your `Dockerfile` should handle building the React application and serving the static files, for example using a multi-stage build with Node.js and Nginx.

Step 3: Core UI and Layout

Design a multi-panel layout. A common pattern is a sidebar for listing snippets and folders, a main content area for the code editor, and a header for controls like search and theme switching. Use `react-router-dom` to manage views for the main dashboard, individual snippets, and the shareable snippet page.

Step 4: Snippet Management and Local Storage

Implement the core CRUD (Create, Read, Update, Delete) functionality for snippets. All snippet data (ID, title, content, language, tags) must be persisted in the browser's `localStorage`. Create custom hooks or services to abstract the `localStorage` interactions.

Step 5: Monaco Editor Integration

Integrate the `@monaco-editor/react` component into your main view. Configure it to display snippet content, apply syntax highlighting

partnr

[Learn More](#)

[Get Started](#)

Step 6: Feature Implementation

Build out the application's features one by one, such as search/filtering, copy-to-clipboard, and theme switching. Refer to the `core_requirements` for the exact specifications of each feature.

Step 7: GitHub Gist API Integration

Implement the functionality to import from and export to GitHub Gists. You will need to make requests to `https://api.github.com/gists`. While full OAuth is not required, your application should be structured to use a personal access token from an environment variable for authenticated requests if available. For this task, anonymous Gist creation is sufficient for the export feature.

Step 8: Environment Configuration

Create a `.env.example` file to document any environment variables your application uses. This is critical for configuration and evaluation.

```
# .env.example
# A GitHub Personal Access Token can optionally be provided for authenticated API requests.
# This is not required for the core functionality but allows for higher rate limits.
REACT_APP_GITHUB_TOKEN=
```

Submission Artifacts Checklist

Ensure your submission includes the following artifacts in your repository's root directory.

- ☐ `README.md` - A comprehensive guide to your project.
- ☐ `docker-compose.yml` - File to run the application via Docker Compose.
- ☐ `Dockerfile` - Defines the container for the web application.

☐ .env.example - Documents all required environment variables.

partnr

[Learn More](#)

[Get Started](#)

Implementation Guidelines

These are advisory suggestions to guide your implementation, not strict requirements.

Architecture and State Management

1. **Component Structure:** Organize your UI into small, reusable components. Consider a structure like `components/layout`, `components/editor`, `components/sidebar`, `components/SnippetList`, etc.
2. **State Management:** For a complex application like this, using a dedicated state management library (like Zustand or Redux Toolkit) is recommended over relying solely on React Context. This helps manage global state like the snippet collection, user settings (theme), and API data more predictably. Use `React Query` or `SWR` for managing the state of API requests to GitHub.
3. **Custom Hooks:** Create custom hooks to encapsulate complex logic, such as `useLocalStorage`, `useDebounce` (for the search input), or `useSnippets`.

Performance and UX

1. **Debounced Search:** To avoid performance issues when searching through snippet content, debounce the search input so that the filter function only runs after the user has stopped typing for a brief period (e.g., 300ms).
2. **Lazy Loading:** The Monaco Editor can be a large bundle. Consider lazy loading the editor component using `React.lazy()` and `Suspense` to improve initial page load time.
3. **Error Handling:** Implement robust error handling for API requests. Display user-friendly notifications if importing or exporting a Gist fails.

Code Quality and Maintainability

1. **Modularity:** Keep your API interaction logic separate from your UI components. Create a dedicated service or module for handling GitHub API calls.
2. **Consistency:** Maintain a consistent coding style throughout the project. Use a linter and formatter like ESLint and Prettier to enforce this.

FAQ

partnr

[Learn More](#)[Get Started](#)

1. A: No, this is a frontend-only application. All data persistence is handled via the browser's `localStorage` as specified in requirement `local-storage-persistence`.

2. Q: Am I required to use a specific state management library?

1. A: No, you are free to choose any state management solution (React Context, Zustand, Redux, etc.). Your choice should be justified by your application's needs.

3. Q: How should I handle the GitHub API token for Gist export?

1. A: The application should be able to create anonymous Gists without a token. However, it should also be designed to optionally use a token from an environment variable (`REACT_APP_GITHUB_TOKEN`) for authenticated requests. This is detailed in requirement `gist-export`.

4. Q: What should the shareable snippet URL look like?

1. A: It should be a client-side route, for example `/snippets/view/:snippetId`. Your application's router should parse the `snippetId` from the URL and display the corresponding snippet in a read-only view, as per requirement `snippet-sharing`.

5. Q: Can I use a different code editor component?

1. A: No, the use of the Monaco Editor is a core part of this task, as specified in requirement `monaco-editor-integration`.

6. Q: How can I programmatically test the copy-to-clipboard functionality?

1. A: Automated testing frameworks like Playwright or Cypress provide APIs to interact with and verify the browser's clipboard content. This is how requirement `copy-to-clipboard` will be verified.

Core Requirements

1.

The application must be fully containerized and runnable with a single command. A `docker-compose.yml` file must be present at

partnr

[Learn More](#)

[Get Started](#)

File: `docker-compose.yml`

Service Name: `web`

Requirements:

1. The `web` service must be built from a `Dockerfile` located in the same repository.
2. It must expose a port (e.g., 3000) on the host machine.
3. It must include a `healthcheck` that verifies the application is running and responsive (e.g., using `curl`).

Verification:

1. Run `docker-compose up --build -d` in the repository root.
2. Verify that the `web` container starts and becomes healthy within 2 minutes.
3. Verify that the application is accessible in a browser at the exposed port.

2. The user must be able to create a new code snippet with a title, content, language, and tags.

UI Elements:

1. A button with `data-testid="create-snippet-button"` to open the creation form/modal.
2. An input for the title: `data-testid="snippet-title-input"` .
3. A select or input for the language: `data-testid="snippet-language-input"` .
4. An input for tags (comma-separated): `data-testid="snippet-tags-input"` .
5. The Monaco editor instance for the code content.
6. A save button: `data-testid="save-snippet-button"` .

Verification:

1. Click `create-snippet-button` .

partnr

[Learn More](#)[Get Started](#)

4. Click `save-snippet-button` .
5. Verify that a new item appears in the snippet list (`data-testid="snippet-list"`) with the correct title.

3. All created snippets must be persistently stored in the browser's `localStorage` .

Storage Key: `code_snippets_data`

Behavior:

1. After creating, updating, or deleting a snippet, the `localStorage` item with the key `code_snippets_data` should be updated.
2. On page reload, the application state (all snippets) must be rehydrated from `localStorage` .

Verification:

1. Create a new snippet.
2. Use developer tools to inspect `localStorage` . Verify that `localStorage.getItem('code_snippets_data')` contains a JSON string representing an array that includes the newly created snippet.
3. Reload the page.
4. Verify the created snippet is still present in the UI.

4. The Monaco Editor must be used to display and edit snippet code, with support for syntax highlighting based on language.

UI Elements:

1. The editor container must have `data-testid="monaco-editor-container"` .

Behavior:

p a r t n r

[Learn More](#)[Get Started](#)

2. The editor's language mode must be set according to the snippet's language property (e.g., javascript, python, go).

Verification:

1. Create a snippet with the language 'javascript' and some JavaScript code.
2. Select the snippet.
3. Verify the editor container exists.
4. Verify that the editor's internal DOM contains elements with classes indicative of JavaScript syntax highlighting (e.g., `mtk6` for keywords like `const`). This confirms the language mode was set correctly.

5. The user must be able to toggle the code editor's theme between a light and a dark mode.

UI Elements:

1. A toggle button/switch with `data-testid="theme-toggle-button"`.

Behavior:

1. Clicking the button toggles the Monaco Editor's theme.
2. The supported themes are `'vs'` (light) and `'vs-dark'` (dark).
3. The selected theme preference should be persisted in `localStorage`.

Verification:

1. Note the initial theme of the editor container (`.monaco-editor`).
2. Click `theme-toggle-button`.
3. Verify the class on the editor container changes (e.g., from `vs` to `vs-dark`).
4. Reload the page and verify the chosen theme is still applied.

p a r t n r

[Learn More](#)[Get Started](#)**UI Elements:**

1. A search input field with `data-testid="search-input"` .
2. The list of snippets container with `data-testid="snippet-list"` .

Behavior:

1. As the user types into the search input, the snippet list is filtered in real-time to show only snippets where the title or content includes the search term (case-insensitive).

Verification:

1. Create two snippets: Snippet A with title "React Hook" and Snippet B with title "Express Middleware".
2. Type "Hook" into the search input.
3. Verify that only Snippet A is visible in the snippet list.
4. Clear the input and verify both snippets are visible again.

7. Users must be able to filter snippets by clicking on a tag.

UI Elements:

1. Tags displayed for each snippet should be clickable elements with `data-testid="snippet-tag"` .

Behavior:

1. Clicking on a tag filters the snippet list to show only snippets that include that tag.
2. A mechanism to clear the tag filter must be present.

Verification:

1. Create Snippet A with tags 'react,auth' and Snippet B with tags 'node,auth'.

partnr

[Learn More](#)[Get Started](#)

4. Clear the filter. Click on a tag element for 'auth'.
5. Verify both Snippet A and B are visible.

8. A button must be available to copy the current snippet's code to the system clipboard.

UI Elements:

1. A copy button with `data-testid="copy-snippet-button"` .

Behavior:

1. When a snippet is being viewed, clicking this button copies the editor's entire content to the clipboard.

Verification:

1. Select a snippet with known content, e.g., `const x = 1;` .
2. Click the `copy-snippet-button` .
3. Programmatically read the content from the browser's clipboard.
4. Verify the clipboard content is exactly `const x = 1;` .

9. The application must be able to generate a shareable, read-only link for a specific snippet.

Routing:

1. A client-side route must be available, e.g., `/snippets/view/:snippetId` .

Behavior:

1. When a user navigates to this URL, the application should display the content of the snippet corresponding to

3. If the `snippetId` is invalid, a "Not Found" message should be displayed.

Verification:

1. Create a snippet and note its unique ID.
2. Navigate directly to `/snippets/view/<snippetId>`.
3. Verify the correct snippet's content is displayed in the editor.
4. Verify that the editor is not editable (e.g., by attempting to type in it programmatically).

10. Users must be able to import a code snippet from a public GitHub Gist URL.

UI Elements:

1. An input field to paste the Gist URL: `data-testid="gist-import-input"`.
2. An import button: `data-testid="gist-import-button"`.

Behavior:

1. The user pastes a Gist URL (e.g., `https://gist.github.com/user/gistId`).
2. Clicking the import button fetches the Gist data from the GitHub API.
3. A new snippet is created using the Gist's description as the title, the file content as the snippet content, and the language from the file type.

Verification:

1. Use a known public Gist URL (e.g., one containing a single `hello.js` file).
2. Paste the URL into the import input.
3. Click the import button.

4. Verify that a new snippet is added to the snippet list with the correct title, content, and language ('javascript').

partnr

[Learn More](#)[Get Started](#)

11. The application must provide a feature to export the current snippet as a new, anonymous public GitHub Gist.

UI Elements:

1. An export button: `data-testid="gist-export-button"` .

Behavior:

1. When viewing a snippet, clicking the export button sends a `POST` request to `https://api.github.com/gists` .
2. The request payload must be correctly formatted to create an anonymous Gist with the snippet's content, title (as description), and language.

Verification:

1. Set up a network request interceptor.
2. Select a snippet to export.
3. Click the `gist-export-button` .
4. Verify that a `POST` request was made to `https://api.github.com/gists` .
5. Inspect the request payload and verify it has the correct structure:

```
{
  "description": "<snippet_title>",
  "public": true,
  "files": {
    "<filename>.<ext>": {
      "content": "<snippet_content>"
    }
  }
}
```

12. Users must be able to back up their entire snippet collection to a JSON file and restore it from one.

partnr

[Learn More](#)

[Get Started](#)

1. An export/backup button: `data-testid="backup-button"` .
2. An import/restore file input: `data-testid="restore-input"` .

Behavior:

1. Clicking the backup button should trigger a browser download of a JSON file containing all snippet data.
2. Selecting a valid backup JSON file via the restore input should overwrite the current snippet collection with the data from the file.

Verification:

1. Create several snippets.
2. Click the `backup-button` . Verify a file download is initiated.
3. Delete all snippets from the UI.
4. Use the `restore-input` to upload the previously downloaded file.
5. Verify that all original snippets are restored and visible in the application.

13. Users must be able to permanently delete a snippet from their collection.

UI Elements:

1. Each snippet in the list must have a delete button: `data-testid="delete-snippet-button"` .

Behavior:

1. Clicking the delete button for a specific snippet removes it from the application state and `localStorage` .

Verification:

1. Count the number of snippets in the list (`data-testid="snippet-list"`).

partnr

[Learn More](#)[Get Started](#)

4. Verify the total count of snippets in the list has decreased by one.
5. Reload the page and verify the snippet remains deleted.

14. The repository must contain a `.env.example` file documenting all environment variables required to run the application.

File Location: `.env.example` (repository root)

Required Content:

1. Must document the `REACT_APP_GITHUB_TOKEN` variable, even if it's optional.
2. Must include any other environment variables your specific framework or setup requires.

Format:

```
REACT_APP_GITHUB_TOKEN=
```

Verification:

1. Check for the existence of the `.env.example` file at the repository root.
2. Parse the file and verify that the `REACT_APP_GITHUB_TOKEN` key is present.

partnr

[Learn More](#)

[Get Started](#)

All rights reserved. Copyright, Partnr 2025-26

